```
print getCount() . PHP_EOL;
print getCount() . PHP_EOL;
```

```
Output:
0
0
```

To the untrained eye, the function `getCount()` might look as if it should initialize an integer at value zero then count upwards in increments of one on each call but returns zero every time. We know better though: the scoping rules dictate that `$counter` is destroyed when execution leaves local scope, so every time `getCount()` is called `$counter` is initialized at zero.

We could get around this with a global variable of course, but there is a much tidier solution to the problem: the keyword `static`, which we have seen previously when we looked at return-by-reference.

A static variable (and therefore its value) will persist even when the scope boundary is crossed, which means it can be used to store data between calls. If we turn `$counter` from above into a static variable, we get a different result:

```
function getCount(): int
{
    static $counter = 0;

    return $counter++;
}

print getCount() . PHP_EOL;
print getCount() . PHP_EOL;
print $counter . PHP_EOL;
```

```
Output:
0
1
Warning: Undefined variable $counter
```

And now our `getCount()` function behaves correctly. When execution leaves the `getCount()` local scope the variable `$counter` and its value is retained. There are two things to notice here. Firstly, PHP does not reinitialize `static $counter` if it already